phase 1 ✓

%rdi = char* input

在栈上开辟空间8字节

0x402400 → %esi = char*

访问对应内存地址

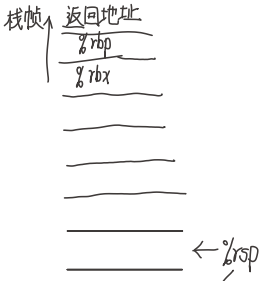
调用 string_not_equals

%rdi → %rbx

%rsi → %rbp

输入对应地址存储的字符串即可

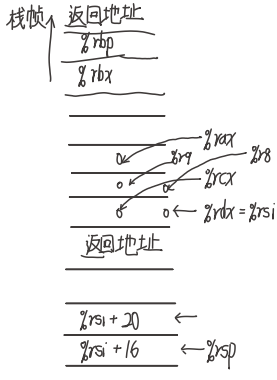
phase 2



%rdi = char* input

%rsi = %rsp

调用 read_six_number



%rdx = %rsi

%rcx = %rsi + 4

%rax = %rsi + 20

%esi = 0x4025C3

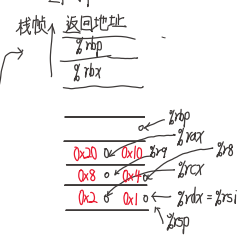
%eax = 0

%rdi = 我输入的串首地址

%rsi = 0x4025C3

%rdx
%rcx
%r8
%r9
} 栈上的四个地址

回到 phase 2



要求 %rdx 指向的内存初始化为 0x1 (四字节)

%rbx = %rcx = %rsp + 4

%rbp = %rsp + 18

%eax = 0x1

%eax = 0x2

要求 %rbx 指向的内存初始化为 0x2 (四字节)

%rbx = %rsp + 8

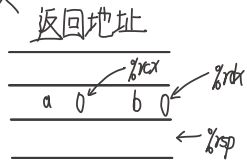
输入 1 2 4 8 16 32

↓

实验程序会将他们写入内存指定位置并进行检查

phase 3

栈帧



输入应为两个四字节的数 b a

b≤7

%eax

未知的内存跳转并修改%eax

%eax = a 时

有多组可行解

b=0 a=0xcf=12×16+15=207

b=1

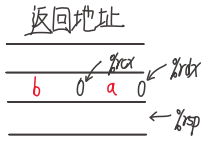
.

b=7

不做赘述

phase 4

栈帧



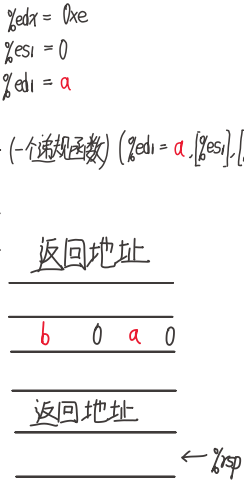
输入为两个四字节的数 a b

要求 a≤14 (e)

%edx = 0xe
%esi = 0
%edi = a

调用fun4 (一个递规函数) (%edi = a, [%esi], [%edx])

栈帧



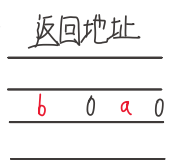

```
%eax = [%edx]
%eax = [%edx] - [%esi]
%ecx = ([%edx] - [%esi]) >> 31   [取该值符号位掩码, 正数时为0, 负数时为-1]   = signmask
%eax = [%edx] - [%esi] + signmask
%eax = ([%edx] - [%esi] + signmask) >> 1
%ecx = %eax + %esi = [esi] + ([%edx] - [%esi] + signmask) >> 1
```

0x7

```
if( %ecx > a) {
    return 2* fun4 (a, [%esi], [%esi] -1 + ([%edx]-[%esi] + signmask) >> 1)
}
else if (%ecx = a) {
        return 0,
    }
else {
    return 2*fun4 (a, [%esi] +1 + ([%edx]-[%esi] + signmask) >> 1, [%edx]) +1
}
```

回到 phase 4

栈帧

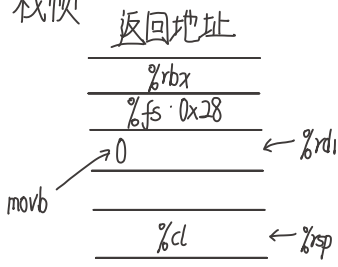


要求fun4 返回值为0，要求 b=0

∴ 输入 a=7, b=0

phase 5

栈帧



低四位

字符串字节应依次为

9 f e 5 6 7

↓

输入

9 ? > 567

%rbx = 输入字符串首地址

%rax = 一个金丝雀数

要求字符串长度必须为6（不含结尾'\0'）

%rax = 0

%ecx = M[%rbx + %rax]　　#按内存顺序取出6个字母

%rdx = %cl　　↓

%rdx　　保留低4位，其余位设为0

- - - -

%edx = M[0x40246D + %rdx]

M[%rsp + %rax + 0x10] = %dl　→ 将栈空间对应字节设为对应数据

%rax = %rax + 1

if %rax != 6

6循环后

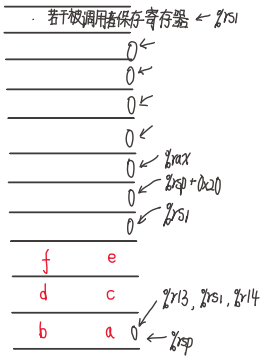
- - - - - -

M[%rsp + 16] = 0

%rsi = 0x40245e

phase 6

栈帧



%r13、%rsi = %rsp

输入应为6个四字节数字 a b c d e f

调用read six number后六个数被写入栈空间

%r14 = %rsp

%r12 = 0

%rbp = %r13 = %rsp

%eax = a − 1

要求 a≤6

存储一个循环计数量

%r12 = 0+1 = 1

if(%r12 != 6)

%ebx = %r12

%rax = %r12

%eax 依次取栈上存储的后5个数

要 b c d e f 和 a 不等

一个双重循环用来判断

a b c d e f 是 ≤6,两两不重复的数

跳出循环

%rsi = %rsp + 0x18

%rax = %rsp

%ecx = 7

%edx = %ecx = 7

%edx -= a

将a b c d e f 的内存位置改变值

改为 7-a ~ 7-f

%esi = 0

%ecx = M[%rsp + %rsi] (7-a ~ 7-f) (读间6个数)

if (7-a ~ 7-f <=1){

%edx = 0x6032d0

M[%rsp + 0x20 + 2*%rsi] = 0x6032d0

}

else {

%eax -1

%edx = 0x6032d0

%rdx = M[0x6032d8]

%eax = 2

if (

依照 7-a ~ 7-f 的值

<=1 8字节位置设为 0x6032d0

=2 M[0x6032d0 + 8]

=3 M[M[0x6032d0+8]+8]

以此类推

%rbx = M[%rsp+0x20]

%rax = %rsp+ 0x28
%rsi = %rsp +0x50 } 指针

%rcx = %rbx

%rdx = M[%rsp+0x28]

循环

M[M[%rsp+0x20]+0x8] = M[%rsp+0x28]

M[M[%rsp+0x28]+0x8] = M[%rsp+0x30]

以此类推

跳出循环

%rbx = M[%rsp+0x20]

M[%rsp+0x48] = 0

%ebp = 5

%rax = M[M[%rsp+0x20]+0x8] = M[%rsp+0x28]

%eax = M[M[%rsp+0x28]]

要求 M[M[%rsp+0x20]] ≥ M[M[%rsp+0x28]]

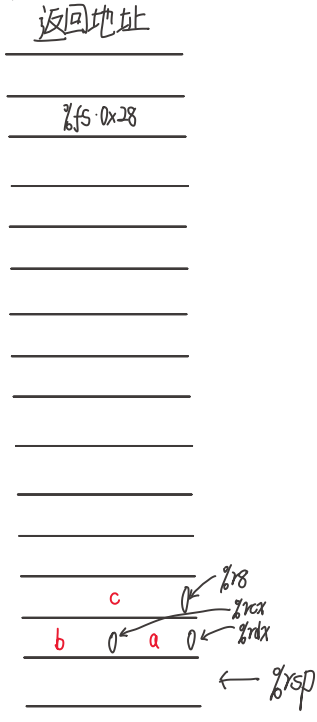
M[M[%rsp+0x28]] ≥ M[M[%rsp+0x30]]

以此类推

进入secret phase

要写在第6行的结尾

栈帧



%edi = 0x603870 → input_string +240的位置开始(第四次输入内容)

%esi = 0x402619 → %d %d %s

第四问

如果输入了额外字符串,进入secret_phase

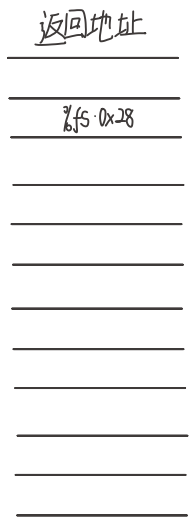
%rsi = 0x402622 "DrEvil"

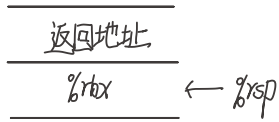
%rdi = %rsp + 0x10

输入的字符串应为

%edi = 0x4024f8

求解 secret phase





%edx = 0x0

%esi = 0

%rdi = %rax = 输入字符串(一个10进制数串)

%rbx = %rax = 转换后的十进制数

%eax = %rax − 1

%eax ≤ 0x3e8 → 输入的数要 ≤ 0x3e9 = 1001

%esi = %ebx = 转换后的十进制数

%edi = 0x6030f0

调用 fun7 → 输出为2就通过

fun7

返回地址

← %rsp

%esi = %ebx = 转换后的十进制数

%edi = 0x6030f0

```
if (%rdi = 空指针) {
    %eax = -1    ret
}
else {
    %edx = M[0x6030f0]   (四字节)
    if (%edx > %esi) {
      %rdi = M[%rdi + 0x8]
      %eax = 2* fun7(%rdi)   ret
    }
    else {
      %eax = 0
      if (%edx = %esi) {
          ret
      }
      else {
          %rdi = M[%rdi + 0x10]   %rax = fun(%rdi)
          %eax = 2* %rax + 1
          ret
      }
```

知道 edx

M[0x6030f0] = 0x24

M[0x6030f8] = 0x603110 %rdi

M[0x603100] = 0x603130

M[0x603110] = 0x8

0x24

8 < %esi < 24

%esi = 0x16 = 22

输入 22